

École Hassania des Travaux Publics  
Département de Mathématiques, Informatique et Géomatique

---

Projet de Programmation en C  
Le Jeu du Moulin  
(Nine Men's Morris)

---

Rapport Technique Détaillé

Réalisé par :

Omar AMDOUNI  
Saoud AMINE  
Hamza AMHIDI

Encadré par :

Mme. Meryem CHERRABI

Année Universitaire 2024-2025

Date : 6 janvier 2026

## Table des matières

|          |                                                      |          |
|----------|------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction Générale</b>                         | <b>3</b> |
| 1.1      | Contexte du Projet . . . . .                         | 3        |
| 1.2      | Objectifs du Projet . . . . .                        | 3        |
| <b>2</b> | <b>Architecture et Structure du Projet</b>           | <b>3</b> |
| 2.1      | Organisation des Fichiers . . . . .                  | 3        |
| 2.2      | Structures de Données Principales . . . . .          | 3        |
| 2.2.1    | Représentation du Plateau . . . . .                  | 3        |
| 2.2.2    | Matrice d'Adjacence . . . . .                        | 4        |
| 2.2.3    | Structure de Mouvement . . . . .                     | 4        |
| <b>3</b> | <b>Implémentation Détaillée des Fonctionnalités</b>  | <b>4</b> |
| 3.1      | Système de Menu Hiérarchique . . . . .               | 4        |
| 3.2      | Gestion des Symboles des Joueurs . . . . .           | 4        |
| 3.3      | Affichage Graphique du Plateau . . . . .             | 5        |
| 3.4      | Détection des Moulins . . . . .                      | 5        |
| 3.4.1    | Alignements Horizontaux . . . . .                    | 5        |
| 3.4.2    | Alignements Verticaux . . . . .                      | 5        |
| 3.5      | Gestion des Phases de Jeu . . . . .                  | 5        |
| 3.5.1    | Phase de Placement . . . . .                         | 5        |
| 3.5.2    | Phase de Mouvement . . . . .                         | 6        |
| <b>4</b> | <b>Implémentation de l'Intelligence Artificielle</b> | <b>6</b> |
| 4.1      | IA Facile (Niveau Débutant) . . . . .                | 6        |
| 4.2      | IA Avancée (Niveau Expert) . . . . .                 | 7        |
| 4.2.1    | Fonction d'Évaluation . . . . .                      | 7        |
| 4.2.2    | Stratégie de Placement Optimale . . . . .            | 7        |
| 4.2.3    | Algorithme de Capture Adaptatif . . . . .            | 7        |
| <b>5</b> | <b>Défis Techniques et Solutions Innovantes</b>      | <b>8</b> |
| 5.1      | Validation Robustes des Entrées . . . . .            | 8        |
| 5.2      | Gestion de la Règle des 3 Pions . . . . .            | 8        |
| 5.3      | Prévention des Conflits de Symboles . . . . .        | 8        |
| <b>6</b> | <b>Stratégie de Tests et Validation</b>              | <b>9</b> |
| 6.1      | Cas de Test Critiques . . . . .                      | 9        |
| 6.2      | Validation des Règles . . . . .                      | 9        |
| <b>7</b> | <b>Performance et Optimisation</b>                   | <b>9</b> |
| 7.1      | Complexité Algorithmique . . . . .                   | 9        |
| 7.2      | Optimisations Implémentées . . . . .                 | 9        |
| <b>8</b> | <b>Conclusion et Perspectives d'Amélioration</b>     | <b>9</b> |
| 8.1      | Bilan du Projet . . . . .                            | 10       |
| 8.2      | Améliorations Futures . . . . .                      | 10       |
| 8.3      | Compétences Développées . . . . .                    | 10       |

---

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>A</b> | <b>Annexe A : Code Source Complet</b> | <b>10</b> |
| <b>B</b> | <b>Annexe B : Guide d'Utilisation</b> | <b>10</b> |
| B.1      | Compilation du Projet . . . . .       | 10        |
| B.2      | Lancement du Jeu . . . . .            | 11        |
| B.3      | Commandes du Jeu . . . . .            | 11        |

# 1 Introduction Générale

Ce rapport présente le projet de programmation en langage C ayant pour objectif l'implémentation du **Jeu du Moulin** (Nine Men's Morris). Ce projet a été réalisé dans le cadre du module de programmation C du département Mathématiques, Informatique et Géomatique de l'École Hassania des Travaux Publics.

## 1.1 Contexte du Projet

Le Jeu du Moulin est un jeu de stratégie antique dont les origines remontent à l'Égypte ancienne (1400 av. J.-C.). Le projet consiste en le développement d'une application console permettant de jouer à ce jeu selon ses règles traditionnelles, avec plusieurs modes de jeu et niveaux d'intelligence artificielle.

## 1.2 Objectifs du Projet

- Implémenter les règles complètes du Jeu du Moulin
- Développer une interface utilisateur conviviale en console
- Créer une intelligence artificielle à deux niveaux de difficulté
- Structurer le code de manière modulaire et maintenable
- Gérer efficacement la logique du jeu et les interactions utilisateur

# 2 Architecture et Structure du Projet

## 2.1 Organisation des Fichiers

Le projet est structuré en plusieurs fichiers source organisés par fonctionnalité :

| Fichier                          | Description                                                 |
|----------------------------------|-------------------------------------------------------------|
| <code>start.c</code>             | Initialisation du jeu, menu principal, affichage du plateau |
| <code>joueurJoueur.c</code>      | Mode de jeu Joueur contre Joueur                            |
| <code>joueurMachine1.c</code>    | Mode de jeu contre IA facile                                |
| <code>joueurMachine2.c</code>    | Mode de jeu contre IA avancée                               |
| <code>fonctions_de_jeu.c</code>  | Fonctions principales du jeu                                |
| <code>fonctions_de_jeu2.c</code> | IA facile (mouvements aléatoires)                           |
| <code>fonctions_de_jeu3.c</code> | IA avancée (stratégies heuristiques)                        |
| <code>fonctions_de_jeu.h</code>  | Fichier d'en-tête avec prototypes et variables globales     |

TABLE 1 – Structure des fichiers du projet

## 2.2 Structures de Données Principales

### 2.2.1 Représentation du Plateau

Le plateau de jeu est modélisé comme un tableau de caractères de taille 24 :

```

1 #define SIZE 24
2 char board[SIZE]; // Chaque case repr sente une intersection

```

Listing 1 – Représentation du plateau de jeu

Les valeurs possibles pour chaque case :

- 'O' : Position vide
- 'X' ou caractère choisi : Pion du Joueur 1
- 'Y' ou caractère choisi : Pion du Joueur 2
- 'm' : Pion de la machine (IA)

### 2.2.2 Matrice d'Adjacence

Une matrice d'adjacence définit les mouvements possibles entre positions :

```

1 int adjacences[SIZE][4] = {
2     {1, 9, -1, -1}, // Position 0 connect e 1 et 9
3     {0, 2, 4, -1}, // Position 1 connect e 0, 2, 4
4     {1, 14, -1, -1}, // Position 2 connect e 1 et 14
5     // ... compl t pour les 24 positions
6 };

```

Listing 2 – Matrice d'adjacence pour le mouvement

### 2.2.3 Structure de Mouvement

Une structure dédiée pour les mouvements de l'IA avancée :

```

1 typedef struct {
2     int from; // Position de d part
3     int to; // Position d'arriv e
4 } Move;

```

Listing 3 – Structure pour les mouvements IA

## 3 Implémentation Détaillée des Fonctionnalités

### 3.1 Système de Menu Hiérarchique

Le jeu propose un menu en trois niveaux :

1. **Menu Principal** : JOUER / Règles / Quitter
2. **Sous-menu Jeu** : Joueur vs Joueur / Joueur vs Machine
3. **Sous-menu Difficulté** : Niveau Débutant / Niveau Avancé

### 3.2 Gestion des Symboles des Joueurs

La fonction `choixSymboles()` gère :

- La validation des caractères alphabétiques
- L'interdiction des symboles réservés ('O', 'm')
- Le tirage au sort du premier joueur

### 3.3 Affichage Graphique du Plateau

L’affichage utilise des codes ANSI pour la coloration :

```
1 void printColoredChar(char c, char joueurSymbol1,
2                       char joueurSymbol2, int position) {
3     if (position == lastMove) {
4         printf("\033[33m%c\033[0m", c); // Jaune
5     } else if (c == joueurSymbol1) {
6         printf("\033[31m%c\033[0m", c); // Rouge
7     } else if (c == joueurSymbol2) {
8         printf("\033[32m%c\033[0m", c); // Vert
9     } else {
10        printf("%c", c); // Normal
11    }
12 }
```

Listing 4 – Fonction d’affichage avec couleurs

### 3.4 Détection des Moulins

La fonction moulin(int i) vérifie deux types d’alignements :

#### 3.4.1 Alignements Horizontaux

```
1 int i1 = i - i % 3; // D but de la ligne
2 if ((board[i1] == board[i1+1]) &&
3     (board[i1] == board[i1+2])) {
4     return 1; // Moulin d tect
5 }
```

Listing 5 – Détection des moulins horizontaux

#### 3.4.2 Alignements Verticaux

```
1 switch (i) {
2     case 0: case 9: case 21:
3         return ((board[0] == board[9]) &&
4                 (board[9] == board[21]));
5     case 1: case 4: case 7:
6         return ((board[4] == board[1]) &&
7                 (board[4] == board[7]));
8     // ... autres configurations verticales
9 }
```

Listing 6 – Détection des moulins verticaux (extrait)

### 3.5 Gestion des Phases de Jeu

#### 3.5.1 Phase de Placement

```

1 void TourDePlacement(char joueurSymbol,
2                     char adversarySymbol,
3                     int joueurNum,
4                     int *adversaryPawns) {
5     // Validation de l'entr e
6     // Placement du pion
7     pose(iplace, joueurSymbol);
8     // V rification de moulin
9     checkAndHandleMoulin(iplace, &itake,
10                          adversarySymbol, adversaryPawns);
11 }

```

Listing 7 – Fonction de placement d'un pion

### 3.5.2 Phase de Mouvement

```

1 int isValidToMove(int ideplace, int* L, int joueurPawns) {
2     if (joueurPawns == 3) {
3         // D placement libre pour 3 pions
4         for (int i = 0; i < SIZE; i++) {
5             if (board[i] == '0') L[count++] = i;
6         }
7     } else {
8         // D placement normal (adjacents)
9         for (int j = 0; j < 4 &&
10              adjacences[ideplace][j] != -1; j++) {
11             int voisin = adjacences[ideplace][j];
12             if (board[voisin] == '0') L[count++] = voisin;
13         }
14     }
15     return count;
16 }

```

Listing 8 – Validation des mouvements

## 4 Implémentation de l'Intelligence Artificielle

### 4.1 IA Facile (Niveau Débutant)

L'IA facile effectue des choix aléatoires mais valides :

```

1 void Machine_placement(char smachine, char shumain,
2                       int *humanPawns) {
3     do {
4         iplace = rand() % 24; // Choix alatoire
5     } while (!isValid(iplace)); // Validation
6     pose(iplace, smachine);
7     checkAndHandleMoulin2(iplace, &itake,
8                          shumain, humanPawns);
9 }

```

Listing 9 – Placement aléatoire de l'IA facile

## 4.2 IA Avancée (Niveau Expert)

L'IA avancée utilise une approche heuristique sophistiquée.

### 4.2.1 Fonction d'Évaluation

```

1 int evaluateBoard(char *board, char machineSymbol,
2                 char humanSymbol) {
3     int score = 0;
4
5     // Points pour les moulins
6     for (int i = 0; i < SIZE; i++) {
7         if (board[i] == machineSymbol && moulin(i))
8             score += 100;
9         if (board[i] == humanSymbol && moulin(i))
10            score -= 100;
11    }
12
13    // Avantage en nombre de pions
14    score += (machinePieces - humanPieces) * 10;
15
16    // Mobilité des pions
17    score += (machineMobility - humanMobility) * 2;
18
19    return score;
20 }
```

Listing 10 – Fonction d'évaluation heuristique

### 4.2.2 Stratégie de Placement Optimale

L'algorithme évalue chaque position potentielle selon :

| Critère                       | Points | Description                          |
|-------------------------------|--------|--------------------------------------|
| Formation immédiate de moulin | +500   | Priorité absolue                     |
| Création de menace            | +50    | Potentiel de moulin au prochain tour |
| Position stratégique          | +30    | Intersections, centres               |
| Blocage adversaire            | +100   | Empêche moulin adverse               |
| Double moulin possible        | +150   | Configuration puissante              |

TABLE 2 – Critères d'évaluation pour le placement

### 4.2.3 Algorithme de Capture Adaptatif

Deux stratégies selon la phase de jeu :

- **Milieu de partie** : Évite de casser les moulins adverses
- **Fin de partie** : Cible spécifiquement les moulins adverses

```

1 int bestcapture1(char *board, char adversarySymbol) {
2     // Milieu de jeu : pr f re pions non-prot g s
3     if (!moulin(i)) score += 200;
4     score += countMobility(i) * 10; // Mobilit
5 }
6
```



```
7 int bestcapture0(char *board, char adversarySymbol) {
8     // Fin de jeu : casse les formations adverses
9     if (moulin(i)) score += 300; // Cible les moulins
10    score -= countMobility(i) * 5; // Isole l'adversaire
11 }
```

Listing 11 – Capture adaptative selon la phase

## 5 Défis Techniques et Solutions Innovantes

### 5.1 Validation Robustes des Entrées

```
1 do {
2     printf("Choisissez une position (0-23) : ");
3     validInput = scanf("%d", &iplace);
4
5     if (validInput != 1) {
6         printf("Erreur : Entrez un nombre !\n");
7         while (getchar() != '\n'); // Nettoyage buffer
8         continue;
9     }
10 } while (!isValid(iplace) || iplace < 0 || iplace > 23);
```

Listing 12 – Solution pour la validation d'entrée

### 5.2 Gestion de la Règle des 3 Pions

Implémentation du déplacement libre :

```
1 if (joueurPawns == 3) {
2     printf("Vous avez 3 pions, vous pouvez sauter !\n");
3     // Toutes les cases vides sont accessibles
4     for (int i = 0; i < SIZE; i++) {
5         if (board[i] == '0') L[count++] = i;
6     }
7 }
```

Listing 13 – Gestion du déplacement libre à 3 pions

### 5.3 Prévention des Conflits de Symboles

```
1 void isDiffMachineAndBoard(char* sj) {
2     while (*sj == 'm' || *sj == '0') {
3         printf("Symbole r serv , choisissez-en un autre : ");
4         scanf(" %c", sj);
5         while (getchar() != '\n');
6     }
7 }
```

Listing 14 – Validation des symboles

## 6 Stratégie de Tests et Validation

### 6.1 Cas de Test Critiques

| Scénario                 | Comportement attendu      | Résultat obtenu |
|--------------------------|---------------------------|-----------------|
| Formation de moulin      | Capture d'un pion adverse | Fonctionnel     |
| Blocage complet          | Victoire automatique      | Fonctionnel     |
| 3 pions restants         | Déplacement libre         | Fonctionnel     |
| Entrée invalide (lettre) | Message d'erreur          | Fonctionnel     |
| Double moulin            | Capture multiple          | Fonctionnel     |

TABLE 3 – Résultats des tests de validation

### 6.2 Validation des Règles

Toutes les règles spécifiées dans le cahier des charges ont été implémentées et testées :

- Placement alterné des 9 pions
- Mouvement vers positions adjacentes
- Formation et détection des moulins
- Capture sélective (sauf pions protégés)
- Règle spéciale des 3 pions
- Conditions de victoire/défaite

## 7 Performance et Optimisation

### 7.1 Complexité Algorithmique

| Fonction                    | Complexité | Optimisation             |
|-----------------------------|------------|--------------------------|
| Détection moulin            | $O(1)$     | Tableaux prédéfinis      |
| Validation mouvement        | $O(1)$     | Matrice d'adjacence      |
| Évaluation IA avancée       | $O(n^2)$   | Heuristiques simplifiées |
| Recherche position optimale | $O(n)$     | Élagage précoce          |

TABLE 4 – Analyse de complexité des fonctions principales

### 7.2 Optimisations Implémentées

- Utilisation de constantes pour les valeurs magiques
- Pré-calcul des configurations de moulins *é Limitation de la profondeur de recherche de l'IA é Gestion efficace de la mémoire (tableaux statiques)*

## 8 Conclusion et Perspectives d'Amélioration

## 8.1 Bilan du Projet

Le projet a atteint tous ses objectifs principaux :

- Implémentation complète des règles du jeu
- Interface console conviviale et colorée
- Deux modes de jeu : Joueur vs Joueur et Joueur vs IA
- Deux niveaux d'IA : Aléatoire et Heuristique
- Gestion robuste des erreurs et validation
- Code modulaire et bien structuré

## 8.2 Améliorations Futures

1. **IA Renforcée** : Implémentation de l'algorithme Minimax avec élagage alpha-bêta
2. **Interface Graphique** : Portage vers SDL pour une expérience visuelle
3. **Fonctionnalités Avancées** :
  - Sauvegarde/chargement des parties
  - Statistiques de jeu
  - Mode tournoi
  - Tutoriel interactif
4. **Optimisations Techniques** :
  - Utilisation de bitboards pour l'état du jeu
  - Tables de transposition pour l'IA
  - Multithreading pour la recherche IA

## 8.3 Compétences Développées

Ce projet a permis de développer des compétences techniques importantes :

- Maîtrise du langage C et de ses bibliothèques standards
- Conception d'algorithmes de jeu et d'IA
- Gestion de projet en équipe
- Débogage et optimisation de code
- Documentation technique professionnelle

## A Annexe A : Code Source Complet

Les fichiers sources complets sont fournis séparément dans l'archive du projet.

## B Annexe B : Guide d'Utilisation

### B.1 Compilation du Projet

```
1 gcc -o jeu_moulin start.c joueurJoueur.c \  
2   joueurMachine1.c joueurMachine2.c \  
3   fonctions_de_jeu.c fonctions_de_jeu2.c \  
4   fonctions_de_jeu3.c -Wall -Wextra
```

Listing 15 – Compilation avec GCC

## B.2 Lancement du Jeu

```
1 ./jeu_moulin
```

## B.3 Commandes du Jeu

- **Menu** : Sélection par numéros (1, 2, 3)
- **Placement** : Entrer un nombre entre 0 et 23
- **Mouvement** : Sélectionner pion puis destination
- **Capture** : Choisir pion adverse après moulin

## Références

- [1] Règles officielles du Jeu du Moulin. *Fédération Internationale des Jeux de Société*, 2023.
- [2] Russell, S., Norvig, P. *Artificial Intelligence : A Modern Approach*. Pearson, 4ème édition, 2020.
- [3] Kernighan, B. W., Ritchie, D. M. *The C Programming Language*. Prentice Hall, 2ème édition, 1988.
- [4] Browne, C. *Modern Techniques for Game AI*. Game Programming Gems, 2001.